AWS Storage on Kubernetes

S3 & EFS CSI Drivers · VPC Private Endpoints

Nginx Deployments · File Locking Tests · S3 Express One Zone

Solutions Architect | Step-by-Step | Complete YAMLs

■■ S3 vs EFS
Object vs POSIX filesystem — why they differ and when to use each

■ File Locking
NFS4 advisory locks on EFS vs no-lock reality of S3 Mountpoint

■ S3 Express
S3 Express One Zone — what it is, cost delta vs standard S3

■ CSI Drivers
aws-efs-csi-driver & aws-mountpoint-s3-csi-driver install & config

■ VPC Endpoints
Interface endpoints for S3 & EFS with private DNS routing

■ Testing
DNS, Flow Logs, flock tests, concurrent write validation

■ Table of Contents

01 · S3 vs EFS — Core Differences

Why You Cannot Use S3 the Same Way as EFS

This is one of the most common architectural misconceptions. S3 is fundamentally an **object store** — a flat key-value system. EFS is a fully POSIX-compliant **network filesystem** built on NFS v4. They solve different problems at different layers.

Feature	Amazon EFS	S3 (Mountpoint)
Protocol	NFS v4.1	FUSE (POSIX-ish)
File Locking	■ NFS4 advisory locks (fcntl/flock)	■ Not supported
Random Writes	■ Full support	■ Not supported
Append Writes	■ Full support	■ Not supported
Atomic Rename	■ POSIX rename()	■■ Limited (overwrite only)
ReadWriteMany	■ Native	■ Yes (read-heavy)
Consistency	■ Strong (NFS)	■ Strong read-after-write
Access Pattern	Files, logs, CMS, shared config	Large objects, ML datasets, backups
Latency	~1-3ms	~10-30ms first byte
Cost (per GB/month)	$0.30 (Standard)	$0.023 (Standard)
Max Throughput	Scales with size	Virtually unlimited
Truncate file	■	■
Directory watch (inotify)	■	■
POSIX permissions	■ Full uid/gid	■■ Limited

File Locking — The Critical Difference

EFS supports **NFS4 advisory file locks** via the standard POSIX *fcntl()* and *flock()* syscalls. Multiple pods can coordinate writes using exclusive locks. S3 Mountpoint has **no locking mechanism** — concurrent writers will silently overwrite each other. This makes S3 unsuitable for databases, CMS platforms, or any workload requiring write coordination.

Lock Type	EFS Behavior	S3 Behavior
flock(LOCK_EX)	Blocks until exclusive lock acquired	ENOTSUP — operation not supported
fcntl(F_SETLK)	POSIX record locking works	Not implemented
fcntl(F_SETLKW)	Blocking wait for lock	Not implemented
Concurrent read	Multiple readers fine	Multiple readers fine
Concurrent write	Serialized via locks	Race condition — last write wins

02 · S3 Express One Zone & Cost

S3 Express One Zone — What AWS Introduced

S3 Express One Zone is NOT a filesystem. It is a high-performance S3 storage class designed for latency-sensitive workloads like ML training, HPC, and analytics. It stores data in a single Availability Zone in purpose-built hardware, achieving **single-digit millisecond latency** and 10x faster data access vs S3 Standard.

Mountpoint for Amazon S3 (the actual filesystem layer)

What actually mounts S3 as a filesystem is **Mountpoint for Amazon S3** — an open-source FUSE-based file client. The **aws-mountpoint-s3-csi-driver** wraps this for Kubernetes. Important limitations to understand:

- ■ Sequential reads — optimized, fast
- ■ New file creation (single-part or multi-part upload)
- ■ Random writes — not supported, writes fail
- ■ Append to existing files — not supported
- ■ File locking (flock/fcntl) — not supported
- ■ Truncate, symlinks, hard links — not supported
- ■■ Best for: read-heavy ML datasets, static assets, log ingestion

Cost Comparison

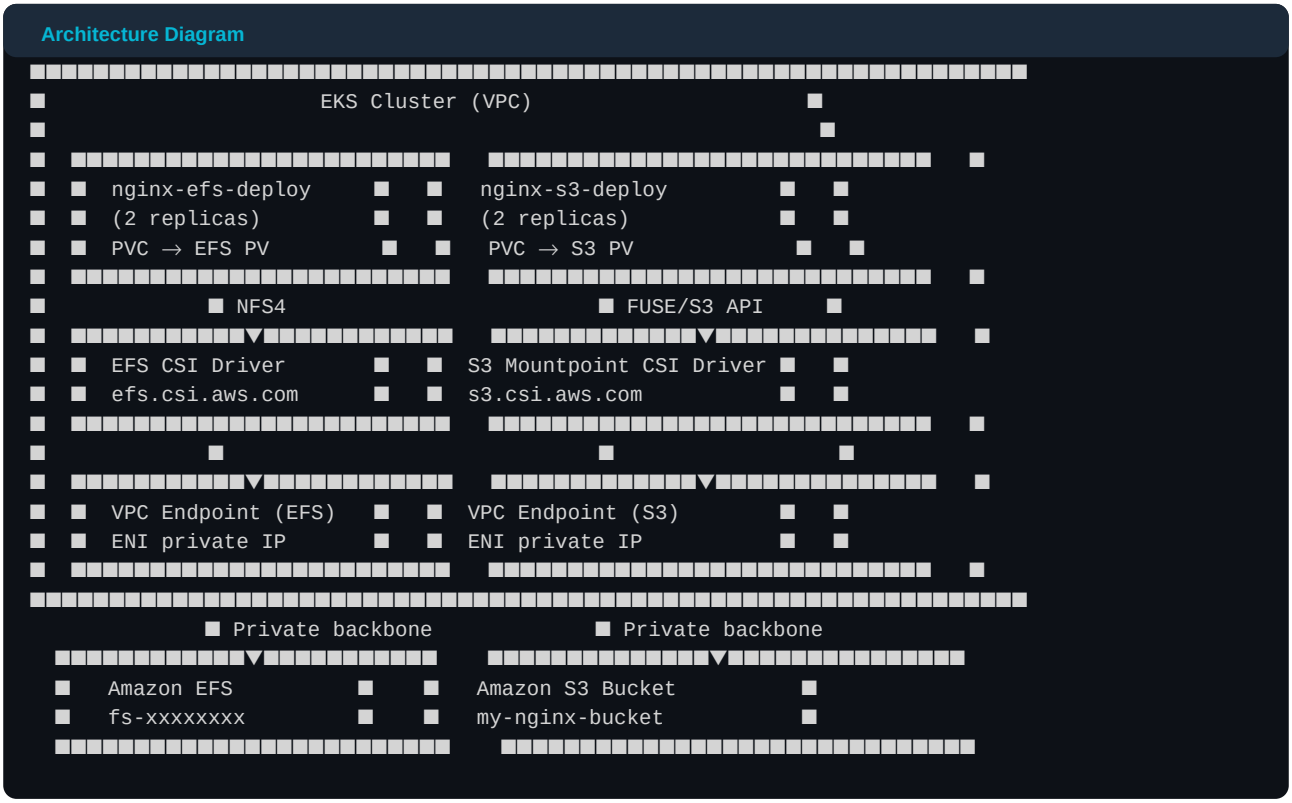
Storage Option	Storage Cost	PUT/GET per 1K	Latency	Use Case
S3 Standard	$0.023/GB	$0.005 / $0.0004	ms–s	General objects
S3 Express One Zone	$0.16/GB	$0.002 / $0.001	<10ms	ML, HPC, low-latency
S3 Intelligent-Tiering	$0.023+/GB	$0.005 / $0.0004	ms–s	Unknown access patterns
EFS Standard	$0.30/GB	—	1-3ms	Shared filesystem
EFS Infrequent Access	$0.016/GB + $0.01/GB retrieval		1-3ms	Infrequently read files
EFS One Zone	$0.043/GB	—	1-3ms	Single-AZ dev workloads

■■ S3 Express One Zone costs ~7x more than S3 Standard for storage. However for request-heavy ML pipelines the lower per-request cost + speed can offset this. EFS remains the right choice for anything needing true POSIX filesystem semantics.

03 · Architecture Overview

End-to-End Flow

Architecture Diagram



Traffic never leaves the VPC. DNS for both efs.ap-south-1.amazonaws.com and s3.ap-south-1.amazonaws.com resolves to ENI private IPs via Route 53 private hosted zones created automatically by the VPC endpoints.

04 · Prerequisites & IAM Setup

Cluster Requirements

- EKS cluster with OIDC provider enabled (required for IRSA)
- Private subnets in at least 2 AZs (for EFS mount targets)
- Worker nodes with IMDSv2 enabled (for metadata-based IAM credentials)
- kubectl, helm, awscli v2 configured on your workstation
- eksctl or Terraform for cluster management

Step 4.1 — Enable OIDC Provider (if not already)

OIDC Provider Setup

```
# Get cluster OIDC issuer
aws eks describe-cluster --name my-cluster --region ap-south-1 \
  --query 'cluster.identity.oidc.issuer' --output text

# Enable OIDC provider via eksctl
eksctl utils associate-iam-oidc-provider \
  --cluster my-cluster --region ap-south-1 --approve

# Verify
aws iam list-open-id-connect-providers | grep $(aws eks describe-cluster \
  --name my-cluster --query 'cluster.identity.oidc.issuer' --output text | cut -d'/' -f5)
```

Step 4.2 — IAM Policy for EFS CSI Driver

EFS IAM Setup

```
# EFS CSI uses the managed policy — attach to node role or IRSA
aws iam attach-role-policy \
  --role-name eks-node-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonEFSCSIDriverPolicy

# Or create IRSA role for EFS CSI driver
eksctl create iamserviceaccount \
  --name efs-csi-controller-sa \
  --namespace kube-system \
  --cluster my-cluster \
  --role-name EFSCSIDriverRole \
  --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEFSCSIDriverPolicy \
  --approve --region ap-south-1
```

Step 4.3 — IAM Policy for S3 CSI Driver

S3 IAM Setup

```
# Create inline S3 Mountpoint policy
cat <<EOF > s3-mountpoint-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "MountpointFullBucketAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject","s3:PutObject","s3:DeleteObject",
        "s3:ListBucket","s3:GetBucketLocation"
      ],
      "Resource": [
        "arn:aws:s3:::my-nginx-bucket",
        "arn:aws:s3:::my-nginx-bucket/*"
      ]
    }
  ]
}
EOF

aws iam create-policy --policy-name S3MountpointPolicy \
  --policy-document file://s3-mountpoint-policy.json

eksctl create iamserviceaccount \
  --name s3-csi-controller-sa \
  --namespace kube-system \
  --cluster my-cluster \
  --role-name S3CSIDriverRole \
  --attach-policy-arn arn:aws:iam::ACCOUNT_ID:policy/S3MountpointPolicy \
  --approve --region ap-south-1
```

05 · VPC Private Endpoints — S3 & EFS

Step 5.1 — Create Security Group for Endpoints

Security Group

```
# Security group allowing NFS (2049) and HTTPS (443) from VPC CIDR
aws ec2 create-security-group \
  --group-name vpc-endpoint-sg \
  --description 'SG for VPC endpoints' \
  --vpc-id vpc-xxxxxxxx

# Allow HTTPS from VPC CIDR (for S3 endpoint)
aws ec2 authorize-security-group-ingress \
  --group-id sg-endpoint-id \
  --protocol tcp --port 443 --cidr 10.0.0.0/16

# Allow NFS from VPC CIDR (for EFS endpoint)
aws ec2 authorize-security-group-ingress \
  --group-id sg-endpoint-id \
  --protocol tcp --port 2049 --cidr 10.0.0.0/16
```

Step 5.2 — Create S3 Interface Endpoint

S3 Endpoint

```
# S3 Interface Endpoint (NOT Gateway — Gateway doesn't support private DNS override)
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-xxxxxxxx \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.ap-south-1.s3 \
  --subnet-ids subnet-private-1a subnet-private-1b \
  --security-group-ids sg-endpoint-id \
  --private-dns-enabled \
  --tag-specifications 'ResourceType=vpc-endpoint,Tags=[{Key=Name,Value=s3-vpc-endpoint}]'

# Verify DNS override is active
# From EC2 inside VPC:
nslookup s3.ap-south-1.amazonaws.com
# Expected: returns 10.x.x.x (ENI private IP)
```

Step 5.3 — Create EFS Interface Endpoint

EFS Endpoint

```
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-xxxxxxxx \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.ap-south-1.elasticfilesystem \
  --subnet-ids subnet-private-1a subnet-private-1b \
  --security-group-ids sg-endpoint-id \
  --private-dns-enabled \
  --tag-specifications 'ResourceType=vpc-endpoint,Tags=[{Key=Name,Value=efs-vpc-endpoint}]'

# Verify
nslookup elasticfilesystem.ap-south-1.amazonaws.com
# Expected: returns 10.x.x.x (ENI private IP)
```

■ Both endpoints use Private DNS. Route 53 private hosted zones override the public service DNS — your EFS CSI driver and S3 Mountpoint driver use the same standard endpoints in their configs; routing is transparent via

DNS.

06 · Create EFS Filesystem & Mount Targets

Step 6.1 — Create EFS Filesystem

Create EFS

```
# Create EFS with encryption
aws efs create-file-system \
  --performance-mode generalPurpose \
  --throughput-mode bursting \
  --encrypted \
  --region ap-south-1 \
  --tags Key=Name,Value=eks-efs-filesystem

# Note the FileSystemId from output: fs-xxxxxxxx
EFS_ID=$(aws efs describe-file-systems \
  --query "FileSystems[?Name=='eks-efs-filesystem'].FileSystemId" \
  --output text)
echo $EFS_ID
```

Step 6.2 — Create Security Group for EFS Mount Targets

EFS Mount Target SG

```
aws ec2 create-security-group \
  --group-name efs-mt-sg \
  --description 'EFS Mount Target SG' \
  --vpc-id vpc-xxxxxxxx

# Allow NFS from worker node SG
aws ec2 authorize-security-group-ingress \
  --group-id sg-efs-mt-id \
  --protocol tcp --port 2049 \
  --source-group sg-worker-node-id
```

Step 6.3 — Create Mount Targets in Each Private Subnet

Mount Targets

```
# Create in each private subnet (one per AZ)
aws efs create-mount-target \
  --file-system-id $EFS_ID \
  --subnet-id subnet-private-1a \
  --security-groups sg-efs-mt-id

aws efs create-mount-target \
  --file-system-id $EFS_ID \
  --subnet-id subnet-private-1b \
  --security-groups sg-efs-mt-id

# Wait for mount targets to be available
aws efs describe-mount-targets \
  --file-system-id $EFS_ID \
  --query 'MountTargets[*].{AZ:AvailabilityZoneName,State:LifeCycleState,IP:IpAddress}'
```

Step 6.4 — Create EFS Access Point

EFS Access Point

```
# Access Point gives each workload an isolated directory
aws efs create-access-point \
  --file-system-id $EFS_ID \
  --posix-user Uid=1000,Gid=1000 \
  --root-directory 'Path=/nginx,CreationInfo={OwnerUid=1000,OwnerGid=1000,Permissions=755}' \
  --tags Key=Name,Value=nginx-access-point

# Note AccessPointId: fsap-xxxxxxxx
```

07 · Install EFS CSI Driver & Kubernetes Resources

Step 7.1 — Install EFS CSI Driver via Helm

EFS CSI Helm Install

```
helm repo add aws-efs-csi-driver https://kubernetes-sigs.github.io/aws-efs-csi-driver/
helm repo update

helm install aws-efs-csi-driver aws-efs-csi-driver/aws-efs-csi-driver \
  --namespace kube-system \
  --set image.repository=602401143452.dkr.ecr.ap-south-1.amazonaws.com/eks/aws-efs-csi-driver \
  --set controller.serviceAccount.annotations.'eks\.amazonaws\.com/role-arn'=arn:aws:iam::ACCOUNT:role/EFSCSID
  --set node.serviceAccount.annotations.'eks\.amazonaws\.com/role-arn'=arn:aws:iam::ACCOUNT:role/EFSCSIDriverR

# Verify pods are running
kubectl get pods -n kube-system -l 'app.kubernetes.io/name=aws-efs-csi-driver'
```

Step 7.2 — StorageClass for EFS (Dynamic Provisioning)

efs-storageclass.yaml

```
# efs-storageclass.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: efs-sc
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap
  fileSystemId: fs-xxxxxxxx    # <-- replace with your EFS ID
  directoryPerClaim: 'true'
  basePath: /dynamic
  uid: '1000'
  gid: '1000'
  gidRangeStart: '1000'
  gidRangeEnd: '2000'
mountOptions:
  - tls
  - iam
volumeBindingMode: Immediate
reclaimPolicy: Retain
allowVolumeExpansion: true
```

Step 7.3 — PersistentVolumeClaim for EFS

efs-pvc.yaml

```yaml
# efs-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: efs-nginx-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteMany   # key -- multiple pods can mount simultaneously
  storageClassName: efs-sc
  resources:
    requests:
      storage: 5Gi
```

■ With dynamic provisioning via efs-ap mode, you don't need to create a PersistentVolume manually. The CSI driver creates an EFS Access Point per PVC automatically.

08 · Install S3 Mountpoint CSI Driver & Resources

Step 8.1 — Install S3 CSI Driver via Helm

S3 CSI Helm Install

```
helm repo add aws-mountpoint-s3-csi-driver \
  https://awslabs.github.io/mountpoint-s3-csi-driver
helm repo update

helm install aws-mountpoint-s3-csi-driver \
  aws-mountpoint-s3-csi-driver/aws-mountpoint-s3-csi-driver \
  --namespace kube-system \
  --set node.serviceAccount.annotations.'eks\.amazonaws\.com/role-arn'=arn:aws:iam::ACCOUNT:role/S3CSIDriverRo

kubectl get pods -n kube-system -l 'app.kubernetes.io/name=aws-mountpoint-s3-csi-driver'
```

Step 8.2 — Create S3 Bucket

Create S3 Bucket

```
aws s3api create-bucket \
  --bucket my-nginx-bucket \
  --region ap-south-1 \
  --create-bucket-configuration LocationConstraint=ap-south-1

# Block public access
aws s3api put-public-access-block \
  --bucket my-nginx-bucket \
  --public-access-block-configuration \
    'BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true'

# Upload initial index.html
echo '<html><body><h1>Hello from S3 Mountpoint!</h1></body></html>' > /tmp/index.html
aws s3 cp /tmp/index.html s3://my-nginx-bucket/html/index.html
```

Step 8.3 — PersistentVolume for S3 (Static Provisioning)

■■ S3 CSI driver uses static provisioning only — you define the PV manually pointing to your bucket.

s3-pv.yaml

```yaml
# s3-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: s3-nginx-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ''   # empty = no StorageClass (static)
  claimRef:
    namespace: default
    name: s3-nginx-pvc
  csi:
    driver: s3.csi.aws.com
    volumeHandle: s3-nginx-volume-unique-id
    volumeAttributes:
      bucketName: my-nginx-bucket
    mountOptions:
      - allow-delete
      - allow-overwrite
      - region ap-south-1
      - prefix html/    # mount only the html/ prefix
```

s3-pvc.yaml

```yaml
# s3-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: s3-nginx-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: ''
  volumeName: s3-nginx-pv
  resources:
    requests:
      storage: 100Gi
```

09 · Nginx Deployments — Full YAML

Step 9.1 — ConfigMap: HTML content for EFS Deployment

efs-html-configmap.yaml

```yaml
# efs-html-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: efs-html-content
  namespace: default
data:
  index.html: |
    <!DOCTYPE html>
    <html><head>
      <title>EFS Nginx</title>
      <style>body{font-family:sans-serif;background:#0f172a;color:#e2e8f0;
        display:flex;flex-direction:column;align-items:center;padding-top:60px}
        h1{color:#38bdf8} .tag{background:#1e40af;padding:4px 12px;border-radius:4px}</style>
    </head><body>
      <h1>Hello from EFS Storage!</h1>
      <p class='tag'>POSIX Filesystem  |  ReadWriteMany  |  File Locking Supported</p>
      <p>Pod: <b>HOSTNAME</b></p>
    </body></html>
  filelock-test.sh: |
    #!/bin/sh
    LOCKFILE=/usr/share/nginx/html/test.lock
    echo 'Acquiring exclusive lock...'
    flock -x $LOCKFILE echo 'Lock acquired by pod: '$(hostname)
```

Step 9.2 — EFS Deployment (2 replicas)

nginx-efs-deployment.yaml

```yaml
# nginx-efs-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-efs
  namespace: default
  labels:
    app: nginx-efs
    storage: efs
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-efs
  template:
    metadata:
      labels:
        app: nginx-efs
        storage: efs
    spec:
      initContainers:
      - name: copy-html
        image: busybox
        command: ['sh','-c','cp /config/* /usr/share/nginx/html/ && chmod 755 /usr/share/nginx/html']
        volumeMounts:
        - name: config-vol
          mountPath: /config
        - name: efs-storage
          mountPath: /usr/share/nginx/html
      containers:
      - name: nginx
        image: nginx:alpine
        ports:
        - containerPort: 80
        volumeMounts:
        - name: efs-storage
          mountPath: /usr/share/nginx/html
        resources:
          requests:
            cpu: 50m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 10
          periodSeconds: 15
      volumes:
      - name: efs-storage
        persistentVolumeClaim:
          claimName: efs-nginx-pvc
      - name: config-vol
        configMap:
          name: efs-html-content
```

Step 9.3 — S3 Deployment (2 replicas)

nginx-s3-deployment.yaml

```yaml
# nginx-s3-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-s3
  namespace: default
  labels:
    app: nginx-s3
    storage: s3
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-s3
  template:
    metadata:
      labels:
        app: nginx-s3
        storage: s3
    spec:
      serviceAccountName: s3-csi-controller-sa  # IRSA
      containers:
      - name: nginx
        image: nginx:alpine
        ports:
        - containerPort: 80
        volumeMounts:
        - name: s3-storage
          mountPath: /usr/share/nginx/html
          readOnly: false
        resources:
          requests:
            cpu: 50m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
        livenessProbe:
          httpGet:
            path: /index.html
            port: 80
          initialDelaySeconds: 15
          periodSeconds: 20
      volumes:
      - name: s3-storage
        persistentVolumeClaim:
          claimName: s3-nginx-pvc
```

Step 9.4 — Services for Both Deployments

services.yaml

```yaml
# services.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-efs-svc
  namespace: default
spec:
  selector:
    app: nginx-efs
  ports:
  - port: 80
    targetPort: 80
  type: ClusterIP
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-s3-svc
  namespace: default
spec:
  selector:
    app: nginx-s3
  ports:
  - port: 80
    targetPort: 80
  type: ClusterIP
```

Deploy everything in order:

Apply all resources

```
kubectl apply -f efs-storageclass.yaml
kubectl apply -f efs-pvc.yaml
kubectl apply -f s3-pv.yaml
kubectl apply -f s3-pvc.yaml
kubectl apply -f efs-html-configmap.yaml
kubectl apply -f nginx-efs-deployment.yaml
kubectl apply -f nginx-s3-deployment.yaml
kubectl apply -f services.yaml

# Verify all pods running
kubectl get pods -l 'app in (nginx-efs,nginx-s3)' -o wide
kubectl get pvc
```

10 · Testing — File Locking & Behavior

Test 1 — VPC Endpoint DNS Verification

DNS Test

```
# Exec into any pod and verify DNS resolves to private IP
kubectl exec -it $(kubectl get pod -l app=nginx-efs -o jsonpath='{.items[0].metadata.name}') -- sh

# Inside pod:
nslookup elasticfilesystem.ap-south-1.amazonaws.com
# Must return 10.x.x.x  -- if public IP, endpoint DNS is broken

nslookup s3.ap-south-1.amazonaws.com
# Must return 10.x.x.x
```

Test 2 — EFS File Locking (flock test)

EFS File Lock Test

```
# Get pod names
EFS_POD1=$(kubectl get pod -l app=nginx-efs -o jsonpath='{.items[0].metadata.name}')
EFS_POD2=$(kubectl get pod -l app=nginx-efs -o jsonpath='{.items[1].metadata.name}')

# From pod1 — acquire exclusive lock, hold for 30 seconds
kubectl exec $EFS_POD1 -- sh -c \
  'flock -x /usr/share/nginx/html/test.lock \
   sh -c "echo locked_by:pod1 && sleep 30"' &

# Immediately from pod2 — try to acquire same lock (should BLOCK)
kubectl exec $EFS_POD2 -- sh -c \
  'flock -x /usr/share/nginx/html/test.lock \
   sh -c "echo locked_by:pod2"'

# Expected behavior:
# pod2 command BLOCKS until pod1 releases (after 30s)
# This proves EFS NFS4 file locking works across pods
```

Test 3 — S3 File Locking Limitation (Expected to Fail)

S3 Lock Limitation Test

```
S3_POD1=$(kubectl get pod -l app=nginx-s3 -o jsonpath='{.items[0].metadata.name}')

# Try flock on S3 mount — WILL FAIL with ENOTSUP
kubectl exec $S3_POD1 -- sh -c \
  'flock -x /usr/share/nginx/html/test.lock echo test 2>&1'

# Expected output:
# flock: /usr/share/nginx/html/test.lock: Operation not supported
# This confirms S3 Mountpoint does NOT support file locking
```

Test 4 — Concurrent Write Test on EFS vs S3

Concurrent Write Test

```
# EFS — concurrent writes serialized by lock (safe)
for i in 1 2; do
  POD=$(kubectl get pod -l app=nginx-efs -o jsonpath="{.items[$((i-1))].metadata.name}")
  kubectl exec $POD -- sh -c \
    'flock -x /usr/share/nginx/html/counter.lock \
     sh -c "echo $(cat /usr/share/nginx/html/count 2>/dev/null || echo 0) | \
     awk '{print $1+1}' > /usr/share/nginx/html/count"' &
done
wait
kubectl exec $EFS_POD1 -- cat /usr/share/nginx/html/count
# Expected: 2  (both writes serialized correctly)

# S3 — concurrent writes: last write wins (data loss risk)
for i in 1 2; do
  POD=$(kubectl get pod -l app=nginx-s3 -o jsonpath="{.items[$((i-1))].metadata.name}")
  kubectl exec $POD -- sh -c \
    'echo write_from_pod_'$i' > /usr/share/nginx/html/concurrent.txt' &
done
wait
kubectl exec $S3_POD1 -- cat /usr/share/nginx/html/concurrent.txt
# Will show only ONE pod's write (race condition — no locking)
```

Test 5 — Verify Nginx Serving Files from Both Mounts

Nginx Serving Test

```
# Port-forward EFS nginx
kubectl port-forward svc/nginx-efs-svc 8080:80 &
curl http://localhost:8080
# Should return EFS HTML content

# Port-forward S3 nginx
kubectl port-forward svc/nginx-s3-svc 8081:80 &
curl http://localhost:8081/index.html
# Should return S3 HTML content

# Test file created by pod1 is visible in pod2 (shared storage proof)
kubectl exec $EFS_POD1 -- sh -c 'echo shared_content > /usr/share/nginx/html/shared.txt'
kubectl exec $EFS_POD2 -- cat /usr/share/nginx/html/shared.txt
# Expected: shared_content
```

Test 6 — S3 Random Write Limitation

S3 Write Limitation Test

```
S3_POD=$(kubectl get pod -l app=nginx-s3 -o jsonpath='{.items[0].metadata.name}')

# Try to append to existing file — will fail
kubectl exec $S3_POD -- sh -c \
  'echo appended >> /usr/share/nginx/html/index.html 2>&1'
# Expected: Operation not supported / Input/output error

# New file creation works fine
kubectl exec $S3_POD -- sh -c \
  'echo newfile > /usr/share/nginx/html/newfile.html'
# This works -- Mountpoint supports creating new files

# Verify new file in S3
aws s3 ls s3://my-nginx-bucket/html/
```

Quick Reference — Feature Matrix

Test	EFS Result	S3 Mountpoint Result	Why
File lock (flock)	■ BLOCKS correctly	■ ENOTSUP	NFS4 vs FUSE
ReadWriteMany	■ Both pods read/write	■ Both pods read/write	NFS vs S3 API
Concurrent write (locked)	■ Serialized safely	■ Race condition	No lock support
Shared file visibility	■ Immediate	■ Yes (eventual)	NFS vs S3 consistency
Append to file	■ Works	■ Not supported	Object immutability
Random write	■ Works	■ Not supported	Object immutability
New file creation	■ Works	■ Works	Both support
Delete file	■ Works	■ Works (allow-delete flag)	Flag required for S3
DNS → private IP	■ 10.x.x.x	■ 10.x.x.x	Both via VPC endpoint